

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Noor Muhammad Akmal Sulaiman NIM. 2410817210007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Muhammad Akmal Sulaiman
NIM : 2410817210007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	17
C. Pembahasan.....	21
SOAL 2	26
A. Pembahasan.....	26
ADDITIONAL	27
A. Pembahasan.....	27
B. Daftar Pustaka	30
TAUTAN GIT.....	31

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi	6
Gambar 2. Tampilan Pilihan Persentase Tip.....	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan.....	7
Gambar 4. Screenshot Tampilan Awal Hasil Jawaban Soal 1 Compose.....	17
Gambar 5. Screenshot Persentase Tip Hasil Jawaban Soal 1 Compose	18
Gambar 6. Screenshot Besar Tip Hasil Jawaban Soal 1 Compose	18
Gambar 7. Screenshot Pembulatan Tip Hasil Jawaban Soal 1 Compose	19
Gambar 8. Screenshot Tampilan Awal Hasil Jawaban Soal 1 XML	19
Gambar 9. Screenshot Persentase Tip Hasil Jawaban Soal 1 XML.....	20
Gambar 10. Screenshot Besar Tip Hasil Jawaban Soal 1 XML	20
Gambar 11. Screenshot Pembulatan Tip Hasil Jawaban Soal 1 XML.....	21

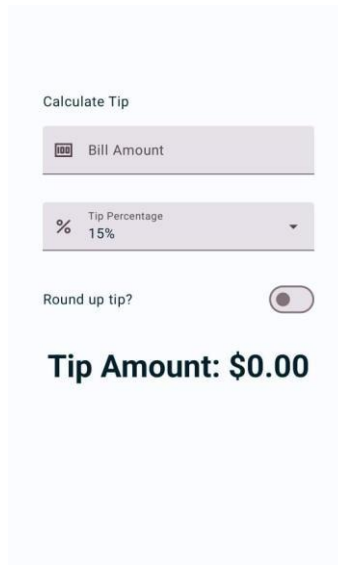
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose	7
Tabel 2. Source Code MainActivity.kt XML.....	13
Tabel 3. Source Code activity_main.xml XML	15
Tabel 4. Test Case Java.....	28
Tabel 5. Test Case Kotlin tanpa Type Inference	28
Tabel 6. Test Case Kotlin dengan Type Inference	29
Tabel 7. Test Case Variabel tanpa Inisialisasi di Kotlin.....	29
Tabel 8. Test Case Type Inference pada Fungsi di Kotlin.....	29

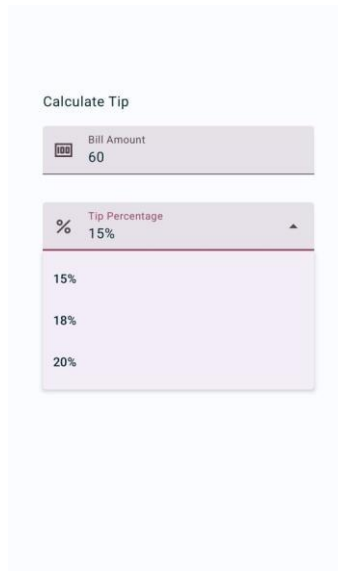
SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

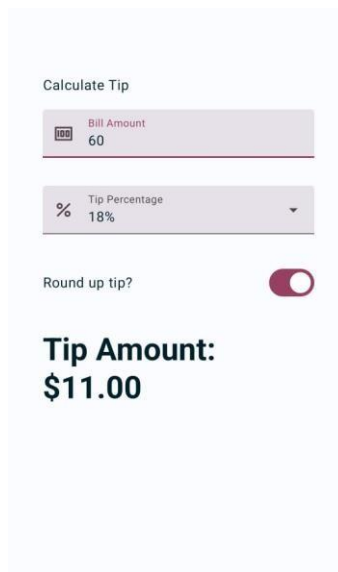
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt (Jetpack Compose)

Tabel 1. Source Code MainActivity.kt Compose

1	package io.github.orizynpx.tipcalculatorcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent

6	import androidx.activity.enableEdgeToEdge
7	import androidx.annotation.DrawableRes
8	import androidx.annotation.StringRes
9	import androidx.compose.foundation.layout.Arrangement
10	import androidx.compose.foundation.layout.Column
11	import androidx.compose.foundation.layout.Row
12	import androidx.compose.foundation.layout.Spacer
13	import androidx.compose.foundation.layout.fillMaxSize
14	import androidx.compose.foundation.layout.fillMaxWidth
15	import androidx.compose.foundation.layout.height
16	import androidx.compose.foundation.layout.padding
17	import androidx.compose.foundation.layout.safeDrawingPadding
18	import androidx.compose.foundation.layout.statusBarsPadding
19	import androidx.compose.foundation.layout.wrapContentWidth
20	import androidx.compose.foundation.rememberScrollState
21	import androidx.compose.foundation.text.KeyboardOptions
22	import androidx.compose.foundation.verticalScroll
23	import androidx.compose.material3.DropdownMenuItem
24	import androidx.compose.material3.ExperimentalMaterial3Api
25	import androidx.compose.material3.ExposedDropdownMenuAnchorType
26	import androidx.compose.material3.ExposedDropdownMenuBox
27	import androidx.compose.material3.ExposedDropdownMenuDefaults
28	import androidx.compose.material3.Icon
29	import androidx.compose.material3.MaterialTheme
30	import androidx.compose.material3.Surface
31	import androidx.compose.material3.Switch
32	import androidx.compose.material3.Text
33	import androidx.compose.material3.TextField
34	import androidx.compose.runtime.Composable
35	import androidx.compose.runtime.getValue
36	import androidx.compose.runtime.mutableStateOf
37	import androidx.compose.runtime.remember
38	import androidx.compose.runtime.setValue
39	import androidx.compose.ui.Alignment
40	import androidx.compose.ui.Modifier
41	import androidx.compose.ui.res.painterResource
42	import androidx.compose.ui.res.stringResource
43	import androidx.compose.ui.text.input.ImeAction
44	import androidx.compose.ui.text.input.KeyboardType
45	import androidx.compose.ui.tooling.preview.Preview
46	import androidx.compose.ui.unit.dp


```

47 import
   io.github.orizynpx.tipcalculatorcompose.ui.theme.TipCalculatorComposeTheme
48 import java.text.NumberFormat
49 import java.util.Locale
50
51 class MainActivity : ComponentActivity() {
52     override fun onCreate(savedInstanceState: Bundle?) {
53         super.onCreate(savedInstanceState)
54         enableEdgeToEdge()
55         setContent {
56             TipCalculatorComposeTheme {
57                 Surface(
58                     modifier = Modifier.fillMaxSize()
59                 ) {
60                     TipCalculatorLayout()
61                 }
62             }
63         }
64     }
65 }
66
67 @Preview(showBackground = true)
68 @Composable
69 fun TipCalculatorLayout() {
70     val option1 = stringResource(R.string.tip_option_1)
71
72     var amountInput by remember { mutableStateOf("") }
73     var tipInput by remember { mutableStateOf(option1) }
74     var roundUp by remember { mutableStateOf(false) }
75
76     val tipPercent = tipInput.replace("%",
77     "").toDoubleOrNull() ?: 0.0
78     val amount = amountInput.toDoubleOrNull() ?: 0.0
79     val tip = calculateTip(amount, tipPercent, roundUp)
80
81     Column(
82         modifier = Modifier
83             .statusBarsPadding()
84             .padding(horizontal = 40.dp)
85             .verticalScroll(rememberScrollState())
86             .safeDrawingPadding(),
87         horizontalAlignment =
88         Alignment.CenterHorizontally,
89         verticalArrangement = Arrangement.Center
90     ) {
91         Text(
92             text =
93             stringResource(R.string.calculate_tip),

```

```

91         modifier = Modifier
92             .padding(bottom = 16.dp, top = 40.dp)
93             .align(alignment = Alignment.Start)
94     )
95     BillNumberField(
96         label = R.string.bill_amount,
97         leadingIcon = R.drawable.money,
98         keyboardOptions =
KeyboardOptions.Default.copy(
99             keyboardType = KeyboardType.Number,
100             imeAction = ImeAction.Next
101         ),
102         value = amountInput,
103         onValueChanged = { amountInput = it },
104         modifier = Modifier
105             .padding(bottom = 32.dp)
106             .fillMaxWidth(),
107     )
108     TipPercentageSpinner(
109         label = R.string.tip_percentage,
110         leadingIcon = R.drawable.percent,
111         selectedValue = tipInput,
112         onValueChanged = { tipInput = it },
113         modifier = Modifier
114     )
115     RoundTipRow(
116         roundUp = roundUp,
117         onRoundUpChanged = { roundUp = it },
118         modifier = Modifier.padding(bottom = 32.dp)
119     )
120     Text(
121         text = stringResource(R.string.tip_amount,
tip),
122         style =
MaterialTheme.typography.displaySmall
123     )
124     Spacer(
125         modifier = Modifier.height(150.dp)
126     )
127 }
128 }
129
130 @Composable
131 fun BillNumberField(
132     @StringRes label: Int,
133     @DrawableRes leadingIcon: Int,
134     keyboardOptions: KeyboardOptions,
135     value: String,
136     onValueChanged: (String) -> Unit,

```

```

137     modifier: Modifier
138 ) {
139     TextField(
140         value = value,
141         leadingIcon = { Icon(painter =
painterResource(id = leadingIcon), null) },
142         singleLine = true,
143         modifier = modifier,
144         onValueChange = onValueChanged,
145         label = { Text(stringResource(label)) },
146         keyboardOptions = keyboardOptions
147     )
148 }
149
150 @OptIn(ExperimentalMaterial3Api::class)
151 @Composable
152 fun TipPercentageSpinner(
153     @StringRes label: Int,
154     @DrawableRes leadingIcon: Int,
155     selectedValue: String,
156     onValueChanged: (String) -> Unit,
157     modifier: Modifier = Modifier
158 ) {
159     var expanded by remember { mutableStateOf(false) }
160     val options = listOf(
161         R.string.tip_option_1,
162         R.string.tip_option_2,
163         R.string.tip_option_3
164     )
165
166     ExposedDropdownMenuBox(
167         expanded = expanded,
168         onExpandedChange = {!expanded},
169         modifier = modifier
170     ) {
171         TextField(
172             value = selectedValue,
173             onValueChange = onValueChanged,
174             readOnly = true,
175             label = { Text(stringResource(label)) },
176             leadingIcon = { Icon(painter =
painterResource(id = leadingIcon), null)},
177             trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(expanded =
expanded) },
178             colors =
ExposedDropdownMenuDefaults.textFieldColors(),
179             modifier = Modifier

```

```

180 .menuAnchor(ExposedDropdownMenuAnchorType.PrimaryNotEdit
181 able, enabled = true)
182     .fillMaxWidth()
183     )
184     ExposedDropdownMenu(
185         expanded = expanded,
186         onDismissRequest = { expanded = false }
187     ) {
188         options.forEach { selectOption ->
189             val option =
190 stringResource(selectOption)
191
192             DropdownMenuItem(
193                 text = { Text(text = option) },
194                 onClick = {
195                     onValueChanged(option)
196                     expanded = false
197                 },
198                 contentPadding =
199 ExposedDropdownMenuDefaults.ItemContentPadding
200             )
201         }
202     }
203 }
204
205 @Composable
206 fun RoundTipRow(
207     roundUp: Boolean,
208     onRoundUpChanged: (Boolean) -> Unit,
209     modifier: Modifier = Modifier
210 ) {
211     Row(
212         modifier = modifier.fillMaxWidth(),
213         verticalAlignment = Alignment.CenterVertically
214     ) {
215         Text(text =
216 stringResource(R.string.round_up_tip))
217         Switch(
218             modifier = Modifier
219                 .fillMaxWidth()
220                 .wrapContentWidth(Alignment.End),
221             checked = roundUp,
222             onCheckedChange = onRoundUpChanged
223         )
224     }
225 }

```

```

224 private fun calculateTip(
225     amount: Double,
226     tipPercent: Double = 15.0,
227     roundUp: Boolean
228 ): String {
229     var tip = tipPercent / 100 * amount
230
231     if (roundUp) {
232         tip = kotlin.math.ceil(tip)
233     }
234
235     return
236     NumberFormat.getCurrencyInstance(Locale.US).format(tip)
237 }

```

MainActivity.kt (XML)

Tabel 2. Source Code MainActivity.kt XML

```

1 package io.github.orizynpx.tipcalculatorxml
2
3 import android.os.Bundle
4 import android.widget.ArrayAdapter
5 import android.widget.AutoCompleteTextView
6 import android.widget.TextView
7 import androidx.activity.enableEdgeToEdge
8 import androidx.appcompat.app.AppCompatActivity
9 import androidx.core.view.ViewCompat
10 import androidx.core.view.WindowInsetsCompat
11 import
12     com.google.android.material.materialswitch.MaterialSwitch
13
14 import
15     com.google.android.material.textfield.TextInputEditText
16 import java.text.NumberFormat
17 import java.util.Locale
18
19 class MainActivity : AppCompatActivity() {
20     private lateinit var numberField: TextInputEditText
21     private lateinit var tipOptions:
22         AutoCompleteTextView
23     private lateinit var roundUpSwitch: MaterialSwitch
24     private lateinit var tipAmount: TextView
25
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         enableEdgeToEdge()
29         setContentView(R.layout.activity_main)

```

```

26 ViewCompat.setOnApplyWindowInsetsListener(findViewById(R
    .id.main)) { v, insets ->
27     val systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars())
28     v.setPadding(systemBars.left,
systemBars.top, systemBars.right, systemBars.bottom)
29     insets
30 }
31
32     numberField = findViewById(R.id.number_field)
33     tipOptions = findViewById(R.id.tip_options)
34     roundUpSwitch =
findViewById(R.id.round_up_switch)
35     tipAmount = findViewById(R.id.tip_amount)
36
37     val tipList = listOf(
38         getString(R.string.tip_option_1),
39         getString(R.string.tip_option_2),
40         getString(R.string.tip_option_3)
41     )
42     val adapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, tipList)
43     tipOptions.setAdapter(adapter)
44     tipOptions.setText(tipList[0], false)
45
46     tipOptions.setOnItemClickListener { _, _, _, _ -
> calculateTip() }
47     roundUpSwitch.setOnCheckedChangeListener { _, _
-> calculateTip() }
48
49     calculateTip()
50 }
51
52     private fun calculateTip() {
53         val billAmount =
numberField.text.toString().toDoubleOrNull() ?: 0.0
54
55         val tipPercentage = when
(tipOptions.text.toString()) {
56             getString(R.string.tip_option_1) -> 0.15
57             getString(R.string.tip_option_2) -> 0.18
58             else -> 0.2
59         }
60
61         var tip = billAmount * tipPercentage
62         if (roundUpSwitch.isChecked) tip =
kotlin.math.ceil(tip)
63

```

64	val formatTip =
65	NumberFormat.getCurrencyInstance(Locale.US).format(tip)
66	tipAmount.text = getString(R.string.tip_amount,
67	formatTip)
	}
	}

activity_main.xml (XML)

Tabel 3. Source Code activity_main.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:orientation="vertical"
9	tools:context=".MainActivity">
10	
11	<TextView
12	android:id="@+id/calculate_tip"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content"
15	android:padding="16dp"
16	android:text="@string/calculate_tip" />
17	
18	
19	<com.google.android.material.textfield.TextInputLayout
20	android:id="@+id/bill_amount"
21	style="@style/Widget.Material3.TextInputLayout.FilledBox"
22	android:layout_width="match_parent"
23	android:layout_height="wrap_content"
24	android:layout_marginBottom="32dp"
25	android:padding="16dp"
26	app:startIconDrawable="@drawable/money">
27	
28	<com.google.android.material.textfield.TextInputEditText
29	android:id="@+id/number_field"
30	android:layout_width="match_parent"
31	android:layout_height="match_parent"
32	android:hint="@string/bill_amount"
33	android:imeOptions="actionDone"
	android:inputType="number"

```

34         android:maxLines="1" />
35
36     </com.google.android.material.textfield.TextInputLayout>
37
38     <com.google.android.material.textfield.TextInputLayout
39         android:id="@+id/tip_percentage"
40
41         style="@style/Widget.Material3.TextInputLayout.FilledBox
42             .ExposedDropdownMenu"
43         android:layout_width="match_parent"
44         android:layout_height="wrap_content"
45         android:layout_marginBottom="32dp"
46         android:padding="16dp"
47         app:startIconDrawable="@drawable/percent">
48
49         <com.google.android.material.textfield.MaterialAutoComple
50         teTextView
51             android:id="@+id/tip_options"
52             android:layout_width="match_parent"
53             android:layout_height="wrap_content"
54             android:focusableInTouchMode="false"
55             android:hint="@string/tip_percentage"
56             android:inputType="none" />
57
58     </com.google.android.material.textfield.TextInputLayout>
59
60     <LinearLayout
61         android:id="@+id/horizontal_main"
62         android:layout_width="match_parent"
63         android:layout_height="wrap_content"
64         android:orientation="horizontal"
65         android:padding="16dp">
66
67         <TextView
68             android:id="@+id/round_tip"
69             android:layout_width="0dp"
70             android:layout_height="wrap_content"
71             android:layout_weight="1"
72             android:text="@string/round_up_tip" />
73
74         <com.google.android.material.materialswitch.MaterialSwit
75         ch
76             android:id="@+id/round_up_switch"
77             android:layout_width="wrap_content"

```



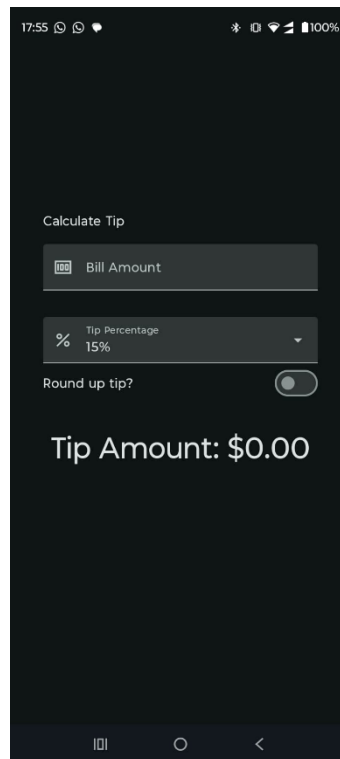
```

74         android:layout_height="wrap_content"
75         android:gravity="center" />
76
77     </LinearLayout>
78
79     <TextView
80         android:id="@+id/tip_amount"
81         android:layout_width="wrap_content"
82         android:layout_height="wrap_content"
83         android:layout_gravity="center"
84         android:layout_marginTop="100dp"
85         android:text="@string/tip_amount"
86         android:textSize="35sp"
87         android:textStyle="bold" />
88
89 </LinearLayout>

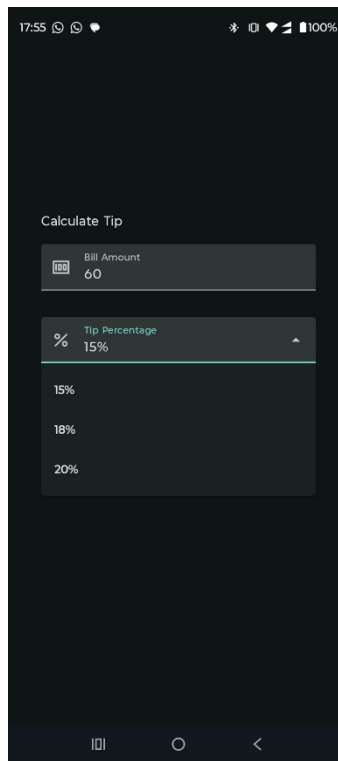
```

B. Output Program

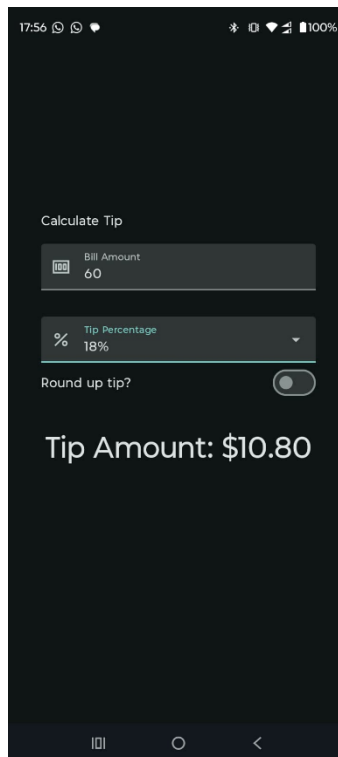
Jetpack Compose



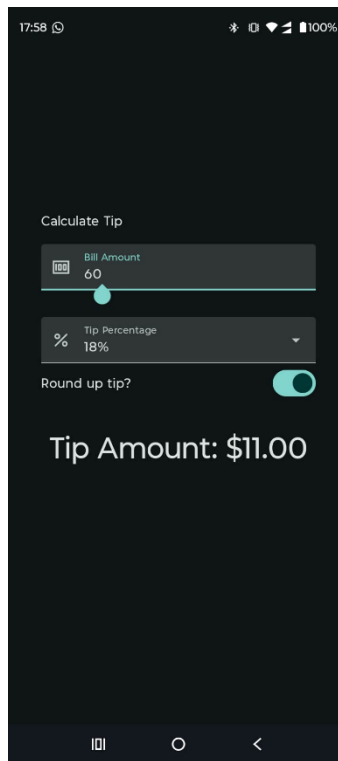
Gambar 4. Screenshot Tampilan Awal Hasil Jawaban Soal 1 Compose



Gambar 5. Screenshot Persentase Tip Hasil Jawaban Soal 1 Compose

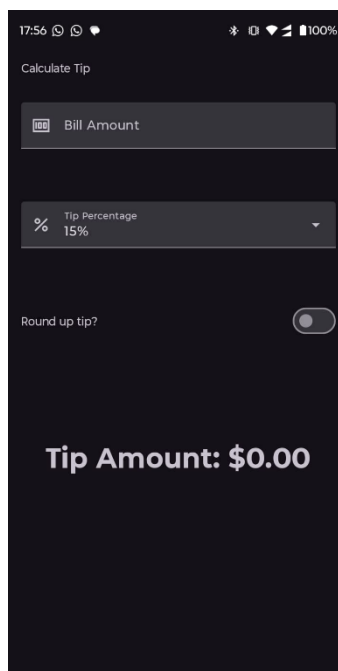


Gambar 6. Screenshot Besar Tip Hasil Jawaban Soal 1 Compose

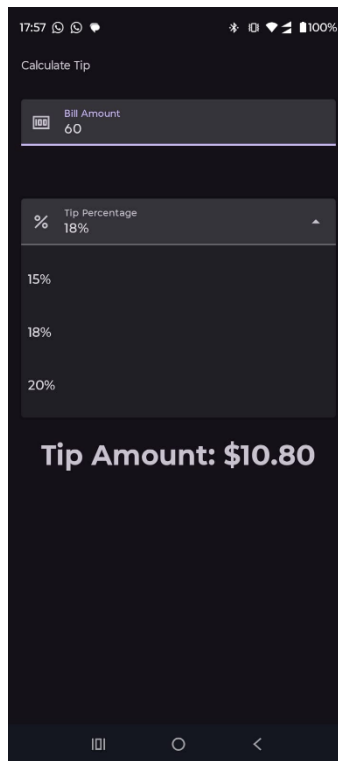


Gambar 7. Screenshot Pembulatan Tip Hasil Jawaban Soal 1 Compose

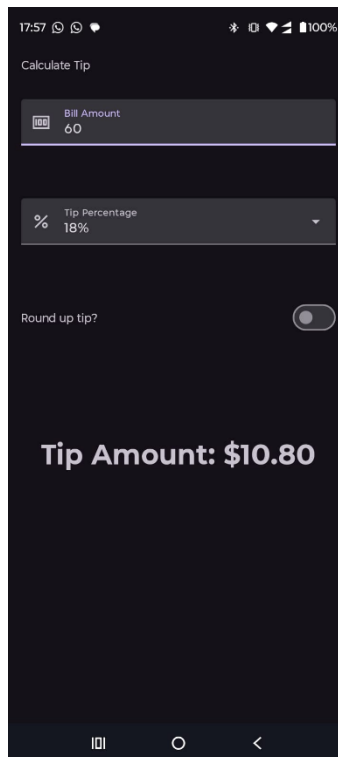
XML



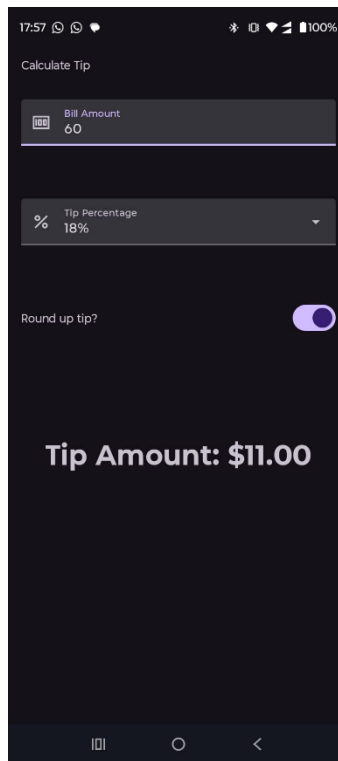
Gambar 8. Screenshot Tampilan Awal Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Persentase Tip Hasil Jawaban Soal 1 XML



Gambar 10. Screenshot Besar Tip Hasil Jawaban Soal 1 XML



Gambar 11. Screenshot Pembulatan Tip Hasil Jawaban Soal 1 XML

C. Pembahasan

MainActivity.kt (Jetpack Compose)

Pernyataan `'package io.github.orizynpx.tipcalculatorcompose'` mendeklarasikan bahwa semua kode dalam *file* ini merupakan bagian dari paket (*package*) dengan nama tersebut.

Blok pernyataan `'import'` digunakan untuk menyertakan berbagai kelas dan fungsi yang dibutuhkan dari *library* Android Jetpack Compose dan *library* standar Java/Kotlin. *Library* ini mencakup komponen UI seperti `'TextField'`, `'Switch'`, dan `'Icon'`, fungsi-fungsi untuk tata letak seperti `'Column'` dan `'Row'`, manajemen *state* seperti `'remember'` dan `'mutableStateOf'`, serta utilitas lain seperti `'NumberFormat'` untuk memformat mata uang.

Kelas `'MainActivity'` adalah kelas utama aplikasi yang mewarisi dari `'ComponentActivity'`. Metode `'onCreate()'` adalah titik masuk utama untuk sebuah *activity* Android. Di dalamnya, `'setContent'` digunakan untuk mendefinisikan UI aplikasi menggunakan Jetpack Compose, di mana seluruh UI dibungkus dalam tema

aplikasi ('TipCalculatorComposeTheme') dan sebuah 'Surface', yang kemudian memanggil *composable* 'TipCalculatorLayout' untuk membangun antarmuka.

Fungsi 'TipCalculatorLayout' adalah *composable* inti yang berisi semua logika dan elemen UI aplikasi. Di dalamnya, beberapa variabel *state* seperti 'amountInput', 'tipInput', dan 'roundUp' dibuat menggunakan 'remember { mutableStateOf(...) }'. Variabel-variabel ini menyimpan input dari pengguna dan akan memicu recomposition (pembaruan UI) setiap kali nilainya berubah. Fungsi ini juga melakukan kalkulasi untuk menghitung jumlah tip dengan memanggil fungsi 'calculateTip'. Tampilan utama dibangun menggunakan 'Column' yang memungkinkan elemen-elemen di dalamnya dapat di-*scroll* secara vertikal. 'Column' ini berisi teks judul, dan memanggil tiga *composable* lain yaitu 'BillNumberField', 'TipPercentageSpinner', dan 'RoundTipRow' untuk membangun *form input*, serta sebuah 'Text' untuk menampilkan hasil perhitungan tip.

Fungsi 'BillNumberField' adalah sebuah *composable* yang dapat digunakan kembali untuk membuat sebuah 'TextField' khusus untuk input angka. Fungsi ini menerima beberapa parameter seperti label, ikon, dan opsi *keyboard* untuk menyesuaikan tampilannya. Di dalam fungsi tersebut terdapat komponen 'TextField' dari Material Design untuk menampilkan *field input*.

Fungsi 'TipPercentageSpinner' adalah sebuah *composable* yang membangun sebuah menu *dropdown (spinner)* untuk memilih persentase tip. Fungsi ini menggunakan komponen 'ExposedDropdownMenuBox' untuk menciptakan efek *dropdown*. Di dalamnya, sebuah 'TextField' yang bersifat *read-only* digunakan untuk menampilkan pilihan yang sedang aktif, dan sebuah 'ExposedDropdownMenu' berisi beberapa 'DropdownMenuItem' yang akan muncul saat 'TextField' tersebut diklik.

Fungsi 'RoundTipRow' adalah sebuah *composable* yang membangun sebuah baris UI yang berisi teks "Round up tip?" dan sebuah komponen 'Switch'. Komponen 'Switch' ini terikat dengan *state* 'roundUp', memungkinkan pengguna untuk mengaktifkan atau menonaktifkan pembulatan tip ke atas.

Fungsi 'calculateTip' adalah fungsi privat yang berisi logika untuk menghitung besaran tip. Fungsi ini menerima jumlah tagihan, persentase tip, dan sebuah *boolean* untuk

pembulatan sebagai input. Setelah menghitung tip, fungsi ini akan membulatkan hasilnya jika diperlukan, dan terakhir memformatnya menjadi *string* mata uang (misalnya “\$1.23”) menggunakan ‘NumberFormat’ sebelum mengembalikannya.

MainActivity.kt (XML)

Pernyataan ‘package io.github.orizynpx.tipcalculatorxml’ mendeklarasikan bahwa semua kode dalam *file* ini merupakan bagian dari paket (*package*) dengan nama tersebut.

Blok pernyataan ‘import’ digunakan untuk menyertakan berbagai kelas yang dibutuhkan dari *library* Android dan Google Material Design. Yang termasuk di dalamnya, yaitu komponen UI seperti ‘ArrayAdapter’, ‘AutoCompleteTextView’, dan ‘TextInputEditText’, kelas ‘AppCompatActivity’ sebagai basis untuk *activity*, serta utilitas lain seperti ‘NumberFormat’ untuk memformat angka, dan lain sebagainya.

Kelas ‘MainActivity’ adalah kelas utama aplikasi yang mewarisi dari ‘AppCompatActivity’. Di dalam kelas ini, beberapa variabel untuk komponen UI seperti ‘numberField’, ‘tipOptions’, ‘roundUpSwitch’, dan ‘tipAmount’ dideklarasikan dengan ‘lateinit’ yang menandakan bahwa nilainya akan diinisialisasi nanti.

Metode ‘onCreate()’ adalah titik masuk utama untuk sebuah *activity* Android. Di dalamnya, ‘setContentView(R.layout.activity_main)’ digunakan untuk menghubungkan kelas ini dengan *file layout* XML-nya. Setelah itu, semua variabel komponen UI diinisialisasi dengan menghubungkannya ke elemen yang sesuai di *file layout* menggunakan ‘findViewById()’. Sebuah ‘ArrayAdapter’ dibuat untuk menampung opsi-opsi persentase tip, yang kemudian dipasang ke ‘tipOptions’ (‘AutoCompleteTextView’) untuk membuat menu *dropdown*. Nilai awal untuk *dropdown* juga diatur. Dua buah *listener*, yaitu ‘setOnItemClickListener’ dan ‘setOnCheckedChangeListener’, dipasang pada *dropdown* tip dan *switch* pembulatan. Kedua *listener* ini akan memanggil fungsi ‘calculateTip()’ setiap kali pengguna memilih opsi baru atau mengubah status *switch*. Terakhir, ‘calculateTip()’ juga dipanggil sekali di akhir ‘onCreate’ untuk menampilkan nilai awal.

Fungsi ‘calculateTip()’ adalah fungsi privat yang berisi logika utama untuk menghitung tip. Ketika dipanggil, fungsi ini pertama-tama mengambil teks dari ‘numberField’, mengubahnya menjadi angka desimal, dan memberikan nilai *default* 0.0 jika input tidak valid. Selanjutnya, sebuah blok ‘when’ digunakan untuk menentukan nilai persentase tip (misalnya 0.15 untuk 15%) berdasarkan teks yang sedang dipilih di ‘tipOptions’. Setelah itu, tip dihitung dengan mengalikan jumlah tagihan dengan persentase tip. Jika ‘roundUpSwitch’ dalam keadaan aktif, hasil perhitungan tip akan dibulatkan ke atas. Terakhir, nilai tip diformat menjadi *string* mata uang menggunakan ‘NumberFormat’, dan hasilnya ditampilkan di komponen ‘TextView’ ‘tipAmount’.

activity_main.xml (XML)

Deklarasi ‘<?xml version="1.0" encoding="utf-8"?>’ di awal *file* adalah standar untuk semua *file* XML.

Tag akar dari *layout* ini adalah ‘<LinearLayout>’, yang merupakan sebuah ViewGroup yang menyusun elemen-elemen di dalamnya secara berurutan. Atribut ‘android:orientation="vertical"' mengatur agar semua elemen anak disusun secara vertikal dari atas ke bawah. Atribut ‘android:id="@+id/main"' memberikan identitas unik pada *layout* ini agar dapat direferensikan dari kode Kotlin, dan android:layout_width serta android:layout_height dengan nilai ‘match_parent’ membuat *layout* ini mengisi seluruh layar.

Di dalam ‘LinearLayout’ utama, terdapat sebuah ‘<TextView>’ yang berfungsi sebagai judul halaman, menampilkan teks yang diambil dari sumber daya *string* ‘@string/calculate_tip’.

Selanjutnya, terdapat dua buah ‘<com.google.android.material.textfield.TextInputLayout>’. Komponen ini berfungsi sebagai pembungkus (*wrapper*) untuk *field* input teks yang memberikan fitur-fitur Material Design seperti *floating label* dan ikon. ‘TextInputLayout’ pertama digunakan untuk input jumlah tagihan. Atribut ‘app:startIconDrawable="@drawable/money"' menambahkan ikon uang di awal *field*. Di dalamnya, terdapat sebuah ‘<com.google.android.material.textfield.TextInputEditText>’ dengan

‘android:id="@+id/number_field"' yang merupakan field input sebenarnya, dengan tipe input diatur hanya untuk angka (‘android:inputType="number"'').

‘TextInputLayout’ kedua digunakan untuk membuat menu *dropdown* (*spinner*) untuk memilih persentase tip, yang ditandai dengan ‘style="@style/Widget.Material3.TextInputLayout.FilledBox.ExposedDropdownMenu"’. Komponen ini juga memiliki ikon di awal. Di dalamnya, terdapat sebuah ‘<com.google.android.material.textfield.MaterialAutoCompleteTextView>’ dengan ‘android:id="@+id/tip_options"' yang berfungsi sebagai tampilan *dropdown*. Atribut ‘android:inputType="none"' mencegah *keyboard* muncul saat diklik.

Setelah itu, terdapat sebuah ‘<LinearLayout>’ lain yang bersifat horizontal (‘android:orientation="horizontal"''). *Layout* ini digunakan untuk menempatkan teks dan *switch* secara berdampingan. Di dalamnya, sebuah ‘<TextView>’ menampilkan teks “Round up tip?”. Di sebelahnya, terdapat sebuah ‘<com.google.android.material.materialswitch.MaterialSwitch>’ dengan ‘android:id="@+id/round_up_switch"' yang berfungsi sebagai tombol geser untuk mengaktifkan atau menonaktifkan fitur pembulatan tip.

Terakhir, di bagian bawah, terdapat sebuah ‘<TextView>’ dengan ‘android:id="@+id/tip_amount"’. Komponen ini digunakan untuk menampilkan hasil akhir dari perhitungan tip. Atribut seperti ‘android:layout_gravity="center"', ‘android:textSize="35sp"', dan ‘android:textStyle="bold"' digunakan untuk memformat teks agar tampil di tengah, berukuran besar, dan tebal.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan utama antara kedua implementasi itu terletak pada paradigmanya. Implementasi XML menggunakan paradigma imperatif. Artinya, *layout* UI-nya didefinisikan secara terpisah dalam *file* XML dan kemudian dimanipulasi secara manual dari kode Kotlin. Anggapannya seperti kita memberi perintah kepada sistem untuk mengubah setiap elemen UI.

Sebaliknya, Jetpack Compose menggunakan paradigma deklaratif, di mana kita mendeskripsikan tampilan UI sebagai fungsi dari data (*state*) langsung di dalam kode Kotlin. Ketika *state* berubah, UI secara otomatis diperbarui oleh *framework*, tanpa perlu diberi tahu secara manual.

Kelebihan utama XML adalah pemisahan yang jelas antara desain dan logika (*separation of concerns*), serta adanya editor visual yang memudahkan kita melihat secara langsung tampilan *layout* yang kita buat, tidak seperti pada Jetpack Compose yang mengharuskan kita untuk menge-*build* proyeknya. Namun, kekurangannya adalah banyaknya kode *boilerplate* (seperti ‘`findViewById`’), manajemen *state* yang sepenuhnya manual, dan risiko ketidaksinkronan antara *file* XML dan kode Kotlin yang dapat menyebabkan *crash* saat *runtime*.

Kelebihan utama Jetpack Compose adalah kodenya jauh lebih ringkas (*concise*) karena tidak memerlukan ‘`findViewById`’. Manajemen *state* di Jetpack Compose bersifat reaktif, artinya UI otomatis merespons perubahan data, sehingga mengurangi kompleksitas dan potensi munculnya *bug*. Jetpack Compose juga sangat memudahkan pembuatan komponen UI yang *reusable*. Namun, kekurangannya adalah *programmer* yang terbiasa dengan XML atau bahkan HTML bisa kesulitan memahami konsep yang ditawarkan Jetpack Compose dan masalah performa bisa muncul jika manajemen *state* tidak diimplementasikan dengan benar.

ADDITIONAL

- Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
- Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Pembahasan

Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

Menurut buku Kotlin in Action, Kotlin terkenal akan sejumlah fitur seperti adanya *data class*, Kotlin DSL (*domain specific language*), *null safety* dalam sistem tipenya (*type system*), *coroutine* untuk pemrosesan asinkron, dan banyak lagi [1]. Kotlin sendiri merupakan bahasa multiparadigma, terutama paradigma *object-oriented programming* (OOP) dan *functional programming* (FP). Hal ini berarti penerapan OOP-nya tidak seketat Java sehingga kita terbebas dari *boilerplate* seperti semua variabel dan fungsi harus ada di dalam kelas atau menulis *setter* dan *getter* secara manual untuk setiap atribut.

Intinya adalah Kotlin dibuat untuk mengatasi kelemahan Java, sehingga “fitur-fitur unggulannya” pun merupakan fitur yang bertujuan mengatasi masalah-masalah seperti *boilerplate* yang saya sebutkan tadi melalui *data class*, sintaks yang ringkas, dan adanya *type inference*, ‘NullPointerException’ yang konon katanya merupakan *billion-dollar mistake* pada Java melalui *nullable types*, dan lemahnya penggunaan *asynchronous programming* melalui *coroutine*. Meskipun Kotlin terdengar seperti bahasa pengganti Java, salah satu “fitur” yang juga ditawarkan adalah *interoperability* dengan Java, yang berarti kode Kotlin bisa digunakan bersamaan dengan Java, karena pada dasarnya kedua bahasa tersebut di-*compile* menjadi *bytecode* pada JVM (Java Virtual Machine).

Menurut saya sendiri, fitur yang sangat-sangat membantu tetapi tidak terlalu dipedulikan (alias fitur yang *underrated*) oleh orang adalah *type inference*. Di Java,

kita harus selalu menandai semua variabel yang kita buat dengan tipe datanya. Hal ini adalah konsekuensi dari sifat bahasanya yang *statically typed*, tetapi karena Java juga merupakan bahasa berparadigma OOP, ujung-ujungnya hal yang terjadi adalah kode seperti ‘Modifier modifier = new Modifier();’. Sementara itu, Kotlin dapat mengurangi kendala seperti itu melalui fitur *type inference*. Jadi, walaupun Kotlin juga merupakan bahasa *statically typed* (tidak seperti Python, JavaScript, atau PHP yang *dynamically typed*), *compiler*-nya bisa mengetahui tipe data variabel kita tanpa kita harus memberitahunya setiap waktu. Penelitian yang dilakukan oleh Flint dkk. (2024) juga menunjukkan bahwa *developer* zaman sekarang juga sangat sering menggunakan fitur *type inference* ini, yang artinya fitur Kotlin tersebut adalah fitur yang populer atau bisa dikatakan merupakan fitur unggulan [2].

Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Misalkan kita ingin membuat dua variabel, satu bertipe data *string*, dan satunya bertipe data *integer*. Dalam bahasa *statically typed* lainnya, umumnya hal tersebut ditulis seperti berikut:

Tabel 4. Test Case Java

1	String name = "Noor Muhammad Akmal Sulaiman";
2	int age = 20;

Di Kotlin, kita juga bisa mendefinisikan variabel seperti itu, hanya saja tipe datanya diletakkan setelah nama variabel, bukan sebelumnya layaknya pada bahasa Java atau C:

Tabel 5. Test Case Kotlin tanpa Type Inference

1	val name: String = "Noor Muhammad Akmal Sulaiman"
2	var age: Int = 20

Namun, hal tersebut dianggap redundan karena baik kita sebagai *programmer* maupun *compiler*-nya sendiri bisa mengetahui apa tipe data variabel-variabel tersebut berdasarkan nilai yang kita berikan. Oleh karena itu, kita bisa menghapus pendefinisian

tipe datanya. Hal ini disebut *type inference* dan merupakan hal yang membuat Kotlin jauh lebih ringkas dibandingkan Java.

Tabel 6. Test Case Kotlin dengan Type Inference

1	val name = "Noor Muhammad Akmal Sulaiman"
2	var age = 20

Walaupun demikian, *compiler* tidak bisa mengetahui tipe data suatu variabel jika kita hanya mendefinisikannya tanpa menginisialisasi nilainya. Maka, kita harus menuliskan tipe datanya.

Tabel 7. Test Case Variabel tanpa Inisialisasi di Kotlin

1	val name: String
2	var age: Int

Hal ini tetap merupakan fitur yang bagus karena kita tidak ingin variabel itu *dynamically typed* karena bisa menyebabkan *bug*. Andaikan saja kita tidak memberi tipe data *string* pada variabel ‘name’ itu, dan ada yang iseng memberinya nilai ‘2410817210007’ (NIM saya), sementara di bagian kode yang lain ada fungsi ‘convertStringToInt(name)’. Alhasil yang terjadi adalah eror karena program tidak menduga bahwa ‘name’ adalah *integer* dan bukan *string*.

Selain hanya variabel, *type inference* juga digunakan untuk menentukan tipe data dari nilai kembalian (*return value*) fungsi. Contohnya:

Tabel 8. Test Case Type Inference pada Fungsi di Kotlin

1	fun max(a: Int, b: Int): Int = if (a > b) a else b // tidak menggunakan type inference
2	fun max(a: Int, b: Int) = if (a > b) a else b // menggunakan type inference

Pada contoh tersebut, ‘a’ dan ‘b’ jelas-jelas diberi tipe data *integer*. Karena fungsinya mengembalikan nilai ‘a’ atau ‘b’ sesuai yang mana yang lebih besar, maka *compiler* dapat mengetahui bahwa tipe data dari nilai kembaliannya juga merupakan *integer*.

B. Daftar Pustaka

- [1] S. Aigner, R. Elizarov, S. Isakova, dan D. Jemerov, *Kotlin in action*, Second edition. Shelter Island, NY: Manning Publications Co, 2024.
- [2] S. W. Flint, A. M. Keshk, R. Dyer, dan H. Bagheri, “How Do Developers Use Type Inference: An Exploratory Study in Kotlin,” 30 Oktober 2024, *arXiv*: arXiv:2410.23360. doi: 10.48550/arXiv.2410.23360.

TAUTAN GIT

<https://github.com/orizynpx/praktikum-pemrograman-mobile/tree/main/Modul%20>